

Lecture 13: The Network Layer

Forwarding, Routing, and the Data/Control Plane

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Explain the role of the network layer and how it differs from the link layer
- Distinguish between the **data plane** and the **control plane**, and explain why this separation matters
- Explain the difference between **forwarding** (local, per-packet) and **routing** (network-wide, algorithmic)
- Describe router architecture at a functional level: input ports, switching fabric, output ports, queuing
- Apply **longest-prefix match** to find the correct forwarding table entry
- Read and interpret a real routing table from the command line
- Explain what SDN is, where it is actually deployed today, and what problem it solves

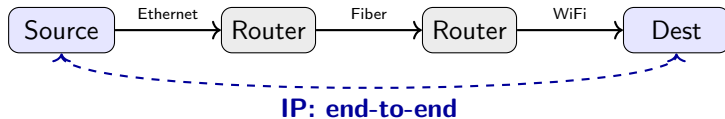
Key Acronyms

Acronym	Meaning	Acronym	Meaning
ASIC	Application-Specific Integrated Circuit	LPM	Longest Prefix Match
BGP	Border Gateway Protocol	OSPF	Open Shortest Path First
DSCP	Differentiated Services Code Point	QoS	Quality of Service
FIB	Forwarding Information Base	RED	Random Early Detection
HOL	Head-of-Line (blocking)	RIB	Routing Information Base
ICMP	Internet Control Message Protocol	SDN	Software-Defined Networking
IP	Internet Protocol	SOHO	Small Office / Home Office
ISP	Internet Service Provider	TCAM	Ternary Content-Addressable Memory
IXP	Internet Exchange Point	TTL	Time to Live

From Links to End-to-End

So far: physical layer and data link layer get frames between *directly connected* nodes — one hop at a time.

The network layer solves the harder problem: deliver a packet from a source host to a destination host *many hops away*, across many different networks.



Key properties of IP:

- **Best-effort:** no delivery guarantee, no ordering, no timing
- **Connectionless:** each packet forwarded independently
- **Universal:** runs on every device, over any link technology

[Wikipedia: Network layer] [Wikipedia: Internet Protocol]

Network Layer: Three Responsibilities

The network layer has three distinct jobs:

1. Addressing

Give every host a globally unique identifier so we can name the destination.

IPv4: 32-bit address

IPv6: 128-bit address

CIDR notation: 10.1.2.0/24

2. Forwarding

At each router: look at incoming packet's destination address, decide which output port to use.

Data plane.

Happens per-packet.

Nanoseconds.

3. Routing

Across the network: compute the paths that forwarding tables should encode.

Control plane.

Dijkstra, Bellman-Ford, BGP.

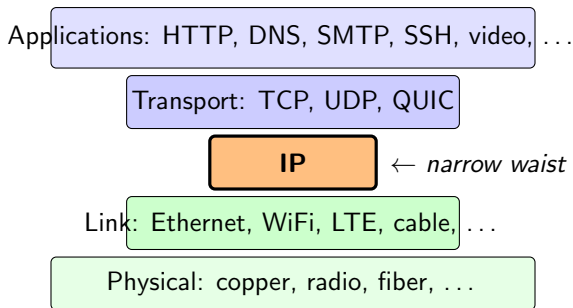
Seconds to minutes.

Two more things you need (not covered today):

- **Getting your IP address:** DHCP assigns addresses automatically when you connect to a network (Lecture 18)
- **Translating names to addresses:** DNS converts google.com → 142.250.80.46 (application layer — later in the course)

IP: The “Narrow Waist”

The internet’s architecture is an **hourglass**: IP is the single protocol everything runs on top of, and that runs on top of everything.



IP’s **simplicity is a feature**: best-effort and connectionless makes it trivial to implement on any new link technology, and lets TCP make its own choices about reliability.

Why Best-Effort? The End-to-End Argument

Best-effort is a deliberate design choice — not an oversight. The theoretical justification is the **end-to-end argument** (Saltzer, Reed, Clark, 1984):

End-to-End Argument (informal)

If a function requires application-specific knowledge to implement correctly, it *must* be implemented at the endpoints. Doing it inside the network is redundant — the application still has to check anyway.

Applied to reliability: even if every router verified delivery, your application still needs to confirm the data arrived correctly *at the right process* and was acted on. So implement reliability once, in TCP, at the endpoints.

Where does reliability actually live?

- **Link layer** (below IP): CRC catches bit errors hop-by-hop; WiFi retransmits lost frames
- **Transport layer** (above IP): TCP provides ordered, reliable, end-to-end byte-stream delivery
- **IP itself:** does nothing — drops corrupt packets, that's it

[Wikipedia: End-to-end principle]

What Best-Effort Does Not Provide: QoS Dimensions

Quality of Service (QoS) describes guarantees an application might need from the network. Best-effort IP provides *none* of these:

Dimension	Meaning	IP best-effort	Where it's handled
Latency	End-to-end delay	Unbounded	CDNs; congestion control
Jitter	Delay <i>variation</i>	Unbounded	Jitter buffers (RTP, QUIC)
Packet loss	% packets dropped	Not guaranteed zero	TCP retransmit; app tolerates
Ordering	Packets arrive in sequence	Not guaranteed	TCP reorders; UDP does not
Throughput	Minimum guaranteed rate	None	TCP fair sharing; DiffServ

Jitter is often more important than latency

A VoIP call with 150 ms *constant* delay sounds fine. A call with 20–200 ms *varying* delay sounds broken. This is why voice/video uses UDP + application-level jitter buffers rather than TCP — TCP's retransmission makes jitter *worse*.

[Wikipedia: Quality of service] [Wikipedia: Jitter]

Best-Effort vs. the Alternatives

This was not the obvious choice. The telephone network provided *guaranteed* reliability at the network level via circuit switching. Two serious competitors tried to bring that model to data:

X.25 (1976–1990s)

- Connection-oriented, reliable packet delivery *at the network layer*
- Dominant in Europe and enterprise networks
- ITU standard; backbone of early internet in many countries
- Expensive per-hop state; slow to deploy new features
- Largely obsolete by mid-1990s

ATM (1980s–1990s)

- Fixed 53-byte cells; hardware-switched
- Quality-of-service guarantees *in the network*
- “The future of networking” ca. 1992
- Extraordinarily complex; expensive
- Lost to IP + faster Ethernet; mostly gone

[Wikipedia: X.25]

[Wikipedia: ATM]

“Good Enough” Beats “Perfect”: A Design Lesson

Why IP/Ethernet won:

- Ran on cheap commodity hardware; cost fell with Moore's Law
- Simple enough to implement quickly on any new link technology
- Failures handled in software at endpoints, not the network
- Open standards + no per-connection state → scales to billions of hosts

Why the complex systems lost:

- Per-connection state in every switch is expensive and fragile
- Standards bodies (ITU, ISO) moved slowly; deployment lagged years
- Guarantees built for voice traffic didn't match data applications
- Once IP had network effects, no migration path existed

The lasting lesson

A simple, “good enough” design that can be deployed and iterated on quickly will often beat a complex, “perfectly engineered” design that takes a decade to standardize. This pattern recurs throughout computing.

Two Jobs, Two Timescales

Every router performs two fundamentally different functions:

Data Plane

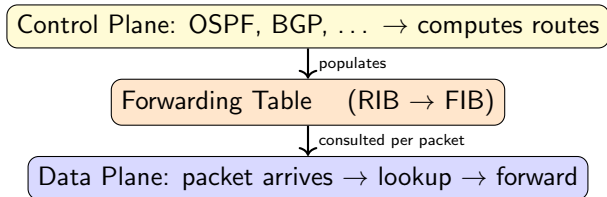
(fast path)

- Per-packet processing
- Nanoseconds to microseconds
- Look at header, look up table, send
- Implemented in **custom hardware** (ASICs)
- Dumb but fast: just executes the table

Control Plane

(slow path)

- Builds and maintains forwarding tables
- Seconds to minutes
- Runs routing protocols (OSPF, BGP)
- Runs on **general-purpose CPU**
- Smart but slow: computes the strategy



[Wikipedia: Forwarding plane]

[Wikipedia: Control plane]

Demo: Reading the Routing Table

Demo: ip route show

The routing table is the **interface** between control plane and data plane.

```
$ ip route show
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.42 metric 100
10.0.0.0/8 via 10.1.0.1 dev eth1 proto static
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.42
```

- default = 0.0.0.0/0 — matches *anything* not more specifically matched
- 10.0.0.0/8 — 10.x.x.x range; installed by static config
- 192.168.1.0/24 scope link — local subnet; directly reachable

Who installed these? Control plane: DHCP client, static config, routing daemon.

Who uses these? Data plane: one lookup per arriving packet.

Also try: `netstat -rn` (macOS/Linux portable)

Forwarding vs. Routing

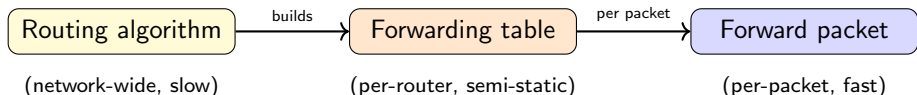
These terms are often used loosely but have precise meanings:

Forwarding

- *Given this packet, which output port?*
- Local to a single router
- Data plane — fast table lookup
- Analogy: following GPS turn-by-turn

Routing

- *What should the tables say?*
- Network-wide computation
- Control plane — Dijkstra, Bellman-Ford, BGP
- Analogy: computing the route before leaving



What Is a Router?

Definition: a router is a device that **forwards packets between networks** at the IP layer (Layer 3) — it connects networks, not just hosts.

Contrast: a *switch* (Layer 2) forwards frames within a single network; a *router* forwards packets *between* networks.

Where routers live:

Router scale and location:

Type	Example	Speed
Home/SOHO	Netgear, TP-Link	1 Gb/s
Campus edge	Cisco ASR 1000	10–100 Gb/s
ISP edge	Cisco ASR 9000	100 Gb/s–1 Tb/s
Tier-1 core	Cisco CRS-X Juniper PTX	1–10+ Tb/s

- **Your home:** WiFi router (also does DHCP, NAT, firewall)
- **BU campus:** edge routers connect BU to the internet; core routers interconnect buildings
- **ISP network:** aggregate traffic from thousands of customers
- **Tier-1 backbone:** major exchange points (MAE-East, Equinix) connecting ISPs globally; no upstream provider

Forwarding: Step by Step

What a router's **data plane** does for each arriving packet:

- ① Receive bits on input interface; reconstruct packet
- ② Check IP header checksum; **decrement TTL**; update checksum
- ③ If $TTL = 0$: **drop** packet, send ICMP Time Exceeded to source
- ④ **Look up** destination IP in forwarding table using longest-prefix match
- ⑤ Send packet out the matching output interface (ARP for next-hop MAC if needed)

TTL: Loop Prevention

Time to Live is decremented at each hop. A packet that gets stuck in a routing loop is eventually dropped rather than circulating forever. Typical initial values: 64 (Linux), 128 (Windows), 255 (Cisco).

[Wikipedia: Time to live]

Demo: Traceroute

Demo: traceroute — watching the hops

Sends packets with $TTL = 1, 2, 3, \dots$. Each router that drops a TTL-expired packet reveals itself via ICMP Time Exceeded.

```
$ traceroute google.com
traceroute to google.com (142.250.80.46), 64 hops max
 1 192.168.1.1 (192.168.1.1) 1.2 ms <- home router
 2 10.24.0.1 (10.24.0.1) 8.4 ms <- ISP edge
 3 72.14.222.1 (72.14.222.1) 14.3 ms <- Google peering
 4 142.251.48.85 (142.251.48.85) 15.1 ms
 5 142.250.80.46 (142.250.80.46) 15.6 ms
```

- Routers are not doing anything special — just executing normal data-plane logic
- * * * at a hop means that router silently drops TTL-expired packets (firewall policy) — *does not mean the path is broken*
- You can estimate hops from source: initial TTL — observed TTL

[Wikipedia: Traceroute]

The Internet Is Smaller Than You Think

Typical traceroute to any destination worldwide: 10–15 hops. Why so few?



- Tier-1 networks (~12 globally) are **directly peered** at internet exchange points (IXPs) — crossing the entire backbone is often **1–2 hops**
- BGP routing selects near-shortest paths; the internet's hierarchy keeps total hop count low by design
- This is a **small-world** property: the graph diameter is ~15 even though the internet has billions of nodes

TTL design implication

TTL = 20 would reach virtually anywhere in the internet. Linux default (64) is $\sim 4\times$ larger than needed — a deliberate safety margin for unexpected routing loops or long paths. Windows (128) and Cisco (255) are even more conservative.

Router Architecture

Input ports (one per physical interface):

- Physical: receive bits, clock recovery
- Link layer: strip header, check FCS
- Lookup destination IP → output port

Switching fabric:

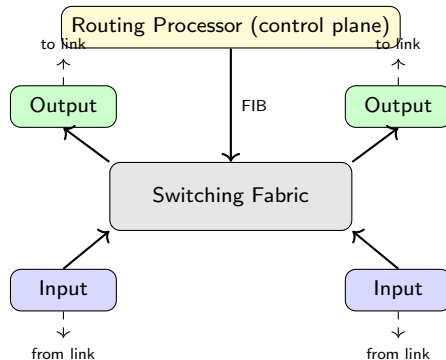
- Interconnects input and output ports
- Must run faster than sum of all input rates

Output ports:

- Buffer packets from switching fabric
- Add link-layer header; transmit

Routing processor (control plane):

- Runs OSPF, BGP
- Builds RIB → installs FIB into line cards



Queuing, Packet Loss, and HOL Blocking

Queues form at output ports when packets arrive from the fabric faster than the link can drain them. When a queue overflows: **packets are dropped** — the fundamental mechanism of congestion.

Drop policies:

- **Tail drop**: drop the newest packet when queue full (default)
- **RED** (Random Early Detection): drop probabilistically *before* queue is full, to signal TCP senders early

Head-of-line (HOL) blocking:

A packet at the head of an input queue waiting for a busy output port *blocks packets behind it* that could go to a free port.

Fix: Virtual Output Queues (VOQ) — separate queue per output port at input.



[Wikipedia: Head-of-line blocking]

[Wikipedia: Random early detection]

Longest Prefix Match

Why LPM? Routing tables store *prefixes* that aggregate many addresses. A destination IP can match multiple entries of different lengths.

Rule: Always use the most specific (longest) matching prefix

The longer the prefix, the more specific the routing instruction.

Prefix	Prefix Length	Next Hop	Interface
0.0.0.0/0	0	203.0.113.1	eth0 (default)
10.0.0.0/8	8	10.255.0.1	eth1
10.1.0.0/16	16	10.1.255.1	eth2
10.1.2.0/24	24	10.1.2.254	eth3

- Destination 10.1.2.5 → matches /0, /8, /16, /24 → **use /24** (eth3)
- Destination 10.5.6.7 → matches /0, /8 → **use /8** (eth1)
- Destination 172.16.0.1 → matches /0 only → **default route** (eth0)

[Wikipedia: Longest prefix match]

Prefix Matching Is Bitwise, Not Byte-Aligned

Prefix length need not be a multiple of 8. The mask boundary can fall *anywhere* within a byte.

Example: 192.168.32.0/20 — the boundary falls at bit 20, mid-way through byte 3:

```
Network: 11000000.10101000.0010|0000.00000000 (192.168.32.0)
Mask /20: 11111111.11111111.1111|0000.00000000
                ^-- bit 20 boundary
```

192.168.33.5 **MATCH**

```
11000000.10101000.0010|0001.00000101
                ^ same 20 bits
```

192.168.48.1 **NO MATCH**

```
11000000.10101000.0011|0000.00000001
                ^^ bits 17-20 differ
```

Key point

192.168.33.x and 192.168.48.x share the same first two *octets*, but fall in *different* /20 networks. Byte boundaries are irrelevant — only the prefix length matters.

Demo: Longest Prefix Match in Python

Demo: ipaddress module (Python stdlib)

```
import ipaddress

def lpm(dest_str, table):
    dest = ipaddress.ip_address(dest_str)
    best = max((p for p in table if dest in p),
               key=lambda p: p.prefixlen)
    n = best.prefixlen
    bits = format(int(dest), '032b')
    # mark the prefix boundary with |
    return best, f"{bits[:n]}|{bits[n:]}"

table = [ipaddress.ip_network(p) for p in
          ["0.0.0.0/0", "10.0.0.0/8", "10.1.0.0/16",
           "10.1.2.0/24", "192.168.32.0/20"]]

for d in ["10.1.2.5", "10.5.6.7", "192.168.33.5", "192.168.48.1"]:
    match, bits = lpm(d, table)
    print(f"{d:16s}-> {str(match):15s} {bits}")
```

Demo: LPM in Python — Output

10.1.2.5	-> 10.1.2.0/24	00001010.00000001.00000010. 00000101
10.5.6.7	-> 10.0.0.0/8	00001010. 00000101.00000110.00000111
192.168.33.5	-> 192.168.32.0/20	11000000.10101000.0010 0001.00000101
192.168.48.1	-> 0.0.0.0/0	11000000.10101000.00110000.00000001

The | marks the prefix boundary — matching bits are to the left.

Observe

- 10.1.2.5 matches four prefixes; the /24 wins (most specific)
- 192.168.33.5 matches the /20 — same first 20 bits as 192.168.32.0
- 192.168.48.1 does *not* match the /20 (bits 17–20 differ) — falls to default

Hardware LPM: TCAM

Problem: a 100 Gb/s link delivers a 1500-byte packet every ~ 120 ns. Sequential search through 900,000 routing table entries is impossible.

Solution: TCAM (Ternary Content-Addressable Memory)

- Stores each entry as a (*value*, *mask*) pair
- Ternary: each bit can be 0, 1, or **X** (don't care) — a natural representation for IP prefix masks
- Searches **all entries in parallel** in a single clock cycle
- Returns the longest matching entry

Tradeoff: TCAM is expensive in power and chip area. The full internet BGP routing table ($\sim 900,000$ prefixes) barely fits in commodity router TCAM — an active engineering constraint.

[Wikipedia: TCAM]

Rethinking the Control Plane

Traditional model: control plane is distributed — OSPF routers exchange link-state advertisements; each runs Dijkstra independently.

Fundamental limitation: each router has only a *local view*. Hard to enforce network-wide policies or optimize global traffic.

The SDN Insight

If the data plane and control plane are cleanly separated, the control plane does not need to run *inside* the router. It can run anywhere — on servers with a **global view** of the entire network.

Traditional:

- Control plane embedded in every router
- Distributed routing protocols
- Policy expressed in complex configs

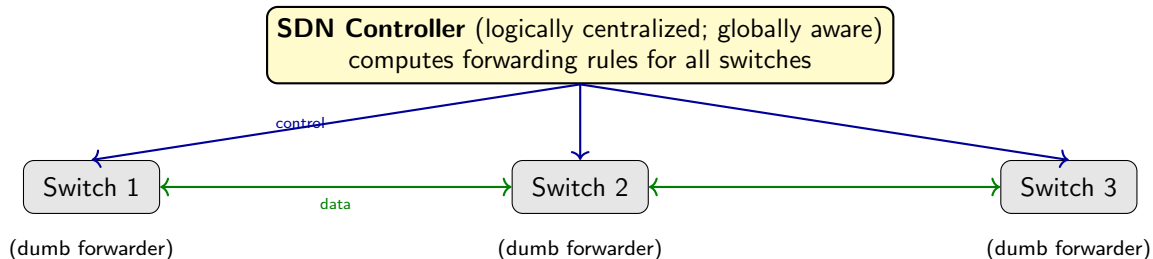
SDN:

- Centralized controller has global topology view
- Pushes forwarding rules to "dumb" switches
- Policy expressed in software

[Wikipedia: SDN]

[Wikipedia: OpenFlow]

SDN Architecture



OpenFlow: controller pushes flow table rules to switches (“if dst IP matches 10.1.2.0/24, send out port 3”). Switches just match and act.

SDN Today: Cloud vs. Open Internet

Where SDN IS the present — cloud data centers:

- **Google B4**: SDN-controlled WAN between data centers; near-100% link utilization vs. 30–40% with traditional OSPF
- **AWS VPC**: networking inside AWS is SDN — controller pushes rules to hypervisors; no traditional routing between VMs
- **Azure, Meta**: similar fabric architectures

Key insight: SDN works where **one organization controls all the infrastructure**.

Where SDN is NOT the answer — open internet:

- BGP and OSPF still run the internet's core
- No single organization can control the whole internet
- “Flag day” style replacement is not feasible

Reality check

SDN was predicted ~2012 to replace all networking. It transformed *cloud* networks; it has not replaced the open internet.

[Wikipedia: SD-WAN / Google B4]

Wireshark: observe the TTL field in a real capture

- 1 Start a capture; ping a distant host or open a webpage
- 2 Filter: `icmp` or `ip.dst == <target>`
- 3 Expand **Internet Protocol** header in the packet details pane
- 4 Find the **Time to Live** field

What to look for:

- TTL = 64 in outgoing packets (Linux default)
- TTL = 54 in reply from `google.com` $\Rightarrow 64 - 54 = 10$ hops
- ICMP Time Exceeded reply: TTL = 1 in the *probe* that triggered it

Connection to traceroute: each line of traceroute output corresponds to one ICMP Time Exceeded reply visible in Wireshark.

What's Next

Today's framework: routers maintain forwarding tables (data plane) built by routing protocols (control plane).

Three open questions:

- ① **L14 — What addresses go in the table?**
IP addressing, CIDR notation, subnetting, address allocation
- ② **L15–L16 — How does the routing algorithm compute optimal paths?**
Link State / Dijkstra; Distance Vector / Bellman-Ford
- ③ **L17 — How do different organizations' routing systems interact?**
Autonomous Systems, BGP, inter-domain routing policy

Big picture

L13 gave you the *vocabulary and architecture*. L14–L18 fill in the *mechanisms*.